

# SmartConsole-to-SolarWinds Integration — Explained

*How the Check Point SmartConsole discovery scripts and the SolarWinds node-creation scripts in this repository fit together.*

---

## 1. What this automation does

The goal is to take the IP addresses of the firewall members that belong to a ClusterXL ("simple-cluster") object in Check Point's Security Management Server, and register each member as a monitored Node in SolarWinds. Doing that reliably means talking to two unrelated systems, each with its own authentication and its own API style, so the work is split into four scripts rather than one large one:

| Script                                                        | Role                                                                                                                                        |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CheckPoint.GetClusterFirewallAddresses.ps1</code>       | Logs into the Check Point Management API (the API behind SmartConsole) and returns the cluster members' IP addresses as PowerShell objects. |
| <code>CRUD.AddNodeWithPollers.ps1</code>                      | A shared helper: adds one SolarWinds Node and registers its default pollers. Contains no Check Point logic at all.                          |
| <code>CRUD.AddNodesFromCsv.ps1</code>                         | Consumer #1 — reads a CSV file (name/IP pairs) and adds each row as a SolarWinds Node.                                                      |
| <code>Orchestrator.AddClusterFirewallsToSolarWinds.ps1</code> | Consumer #2 — calls the Check Point discovery function directly and feeds the results straight into SolarWinds, no file in between.         |

## Why four files instead of one

The Check Point side and the SolarWinds side are kept as two separate, independently reusable pieces (functions you dot-source), rather than one monolithic script. That buys three things:

- **Independent schedules and review.** Discovery can run on its own cadence, and the resulting list can be reviewed before anything is added to monitoring.
- **Independent testing.** Each side can be exercised (or mocked) without needing credentials for the other system.
- **Reuse.** `Add-SwisNodeWithDefaultPollers` doesn't know or care where the IP address came from — it is exactly the same function used by `CRUD.AddNodesFromCsv.ps1` and by the orchestrator.

The two consumer scripts represent the two ends of that trade-off: CSV-based for a manual review gate, orchestrator-based for a single unattended run (e.g. a scheduled task).

### `CheckPoint.GetClusterFirewallAddresses.ps1`

Talks to the Check Point Management API — the REST API (`/web_api/`) that SmartConsole itself is a client of. It exposes three functions plus one that composes them.

## Connect-SmartConsole

Logs in with POST /web\_api/v1.9/login, sending the username and a decrypted-in-memory copy of the SecureString password. The server responds with a session id (sid), which is returned inside a small session object (Server, Port, Sid, TrustAllCertificates) that every later call needs.

```
$body = @{ user = $Username; password = $plainPassword } | ConvertTo-Json
$uri = "https://${Server}:${Port}/web_api/v1.9/login"
$response = Invoke-RestMethod -Uri $uri -Method Post -Body $body -ContentType "application/json"
# $response.sid is the session id used for every subsequent call
```

## Invoke-SmartConsoleApi

A thin wrapper around Invoke-RestMethod that every other API call goes through: it builds the URL from the command name, JSON-encodes the body, and attaches the session id as the X-chkp-sid header the API requires.

## Get-SmartConsoleClusterFirewallAddresses

The actual discovery logic. Check Point's ClusterXL objects are represented by the simple-cluster object type. This function:

- Calls show-simple-clusters to page through every cluster object (50 at a time, following the API's limit/offset/total pagination convention).
- For each cluster, calls show-simple-cluster with details-level=full to get the full member list.
- Flattens the result: one output row per cluster member, carrying the cluster's name and VIP alongside that member's own name and IP address.

```
foreach ($member in $cluster.'cluster-members') {
    $results += [pscustomobject]@{
        ClusterName = $cluster.name
        ClusterVip = $cluster.'ipv4-address'
        MemberName = $member.name
        MemberIp = $member.'ip-address'
    }
}
```

MemberName / MemberIp are exactly the column names the two SolarWinds consumer scripts expect — that shared naming is what lets the CSV handoff and the direct orchestrator handoff both work without a translation step.

## Disconnect-SmartConsole

Calls logout to invalidate the session id. Always call this in a finally block so a failed discovery doesn't leak an open session.

Certificates: by default the script does **not** bypass TLS verification. Check Point management servers commonly present a self-signed certificate, so a -TrustAllCertificates switch is available — but it's opt-in, not the default, because silently trusting any certificate defeats the purpose of TLS.

## CRUD.AddNodeWithPollers.ps1

One function, Add-SwisNodeWithDefaultPollers, factored out of the original CRUD.AddNode.ps1 sample so both consumer scripts below could call it instead of each duplicating the same SWIS calls.

```
function Add-SwisNodeWithDefaultPollers {
```

```

param($Swis, [string]$IPAddress, [string]$Name = "",
      [string]$Community = "public", [string]$RWCommunity = "")

$newNodeProps = @{
    IPAddress = $IPAddress; EngineID = 1; Caption = $Name;
    ObjectSubType = "SNMP"; SNMPVersion = 2;
    Community = $Community; RWCommunity = $RWCommunity;
    DNS = ""; SysName = "";
}
$newNodeUri = New-SwisObject $Swis -EntityType "Orion.Nodes" -Properties $newNodeProps
$nodeProps = Get-SwisObject $Swis -Uri $newNodeUri

foreach ($pollerType in @("N.Status.ICMP.Native", "N.ResponseTime.ICMP.Native",
                        "N.Details.SNMP.Generic", "N.Uptime.SNMP.Generic")) {
    New-SwisObject $Swis -EntityType "Orion.Pollers" -Properties @{
        NetObject = "N:" + $nodeProps["NodeID"]; NetObjectType = "N"
        NetObjectID = $nodeProps["NodeID"]; PollerType = $pollerType
    } | Out-Null
}
$nodeProps
}

```

Community is the SNMP v2 read community string SolarWinds uses to poll the device; it defaults to "public" but is a parameter so real firewalls (which should never be left on the default community) can pass their own. RWCommunity is the optional write community, left blank unless SolarWinds needs to configure the device, not just monitor it.

Two SWIS calls per node (New-SwisObject then Get-SwisObject) are needed because the create call only returns a URI — the NodeID used to register pollers has to be fetched back afterwards.

## CRUD.AddNodesFromCsv.ps1

The "decoupled" consumer. Reads a CSV (by default .\ClusterFirewalls.csv — the file Export-Csv in the SmartConsole script's example usage produces) and calls the shared helper once per row.

```

$rows = Import-Csv -Path $CsvPath
foreach ($row in $rows) {
    $name = $row.$NameColumn # default column: MemberName
    $ip = $row.$IpColumn # default column: MemberIp
    if ([string]::IsNullOrEmpty($ip)) { continue }
    Add-SwisNodeWithDefaultPollers -Swis $swis -Name $name -IPAddress $ip -Community $Community
}

```

The column names (-NameColumn / -IpColumn) are parameters rather than hard-coded, so the same script works against a CSV from any source, not only the Check Point one.

Because the CSV sits between the two systems as a plain file, it doubles as an audit trail: you can open it, delete a row you don't want monitored, or diff it against yesterday's file before running this script.

## Orchestrator.AddClusterFirewallsToSolarWinds.ps1

The "single run" consumer. Dot-sources both function files, then wires them together in memory with no CSV file at all:

```

. (Join-Path $PSScriptRoot "CheckPoint.GetClusterFirewallAddresses.ps1")
. (Join-Path $PSScriptRoot "CRUD.AddNodeWithPollers.ps1")

$scpSession = Connect-SmartConsole -Server $scpServer -Username $scpUsername -Password $scpPassword
try { $firewalls = Get-SmartConsoleClusterFirewallAddresses -Session $scpSession }

```

```

finally { Disconnect-SmartConsole -Session $cpSession }

$swis = Connect-Swis -host $swHostname -cred $swCred
foreach ($fw in $firewalls) {
    Add-SwisNodeWithDefaultPollers -Swis $swis -Name $fw.MemberName `
        -IPAddress $fw.MemberIp -Community $swCommunity
}

```

Dot-sourcing (the leading `.`) runs the target file in the current scope instead of a child scope, so the functions it defines (`Connect-SmartConsole`, `Get-SmartConsoleClusterFirewallAddresses`, `Add-SwisNodeWithDefaultPollers`, ...) become callable here. Neither sourced file executes its own example-usage block, because that block is commented out at the bottom of each file — dot-sourcing only defines the functions.

This script needs both the Check Point credential and the SolarWinds credential available in the same run, and it adds nodes as soon as they're discovered — there is no review step. Use the CSV path instead when that gap matters (e.g. production monitoring, where an unreviewed cluster misconfiguration could otherwise get monitored automatically).

## 6. End-to-end flow, both paths

| Step | Reviewed path (CSV)                                                                              | Unattended path (orchestrator)                |
|------|--------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 1    | Run <code>CheckPoint.GetClusterFirewallAddresses.ps1</code> , <code>Export-Csv</code> the result | Run the orchestrator script                   |
| 2    | Open/review <code>ClusterFirewalls.csv</code>                                                    | (discovery happens in-memory, no review step) |
| 3    | Run <code>CRUD.AddNodesFromCsv.ps1 -CsvPath ClusterFirewalls.csv</code>                          | Nodes are added immediately                   |
| 4    | Nodes + pollers created in SolarWinds                                                            | Nodes + pollers created in SolarWinds         |

### Things to change before running for real

- Fill in real Check Point and SolarWinds hostnames/credentials — every script ships with placeholder values and expects you to edit them (matching the convention of every other script in this repository, e.g. `CRUD.AddNode.ps1`).
- Set a real SNMP community string per environment — "public" is a default, not a recommendation.
- Decide on `-TrustAllCertificates` deliberately if the Check Point management server uses a self-signed certificate, rather than always passing it.

---

Generated to document the scripts under `Scripts/` in this repository: `CheckPoint.GetClusterFirewallAddresses.ps1`, `CRUD.AddNodeWithPollers.ps1`, `CRUD.AddNodesFromCsv.ps1`, and `Orchestrator.AddClusterFirewallsToSolarWinds.ps1`.